

UNIwersytet Śląski w Katowicach

ZBIORY ROZMYTE

Inteligentny sterownik HVAC oparty na logice rozmytej

Autor:
Damian Białek

9 czerwca 2026



Spis treści

1	Wstęp i cel projektu	2
2	Podstawy teoretyczne logiki rozmytej	2
2.1	Pojęcie zbioru rozmytego i funkcji przynależności	2
2.2	Operatory logiczne	2
2.3	Model wnioskowania Mamdaniego i proces wyostrzania	3
3	Projekt struktury sterownika	3
3.1	Definicja przestrzeni rozważań i zmiennych wejściowych	3
3.2	Definicja zmiennych wyjściowych	4
4	Baza wiedzy i reguły	4
4.1	Strategia sterowania w profilu dziennym	5
4.2	Strategia sterowania w profilu nocnym	5
4.3	Algorytmy specjalne: Blokada deszczowa	5
5	Architektura oprogramowania i implementacja	6
5.1	Struktura projektu	6
5.2	Plik inicjalizacyjny jako wzorzec Fasady	6
6	Interfejs użytkownika i moduł diagnostyczny	6
7	Scenariusze testowe i analiza działania systemu	7
7.1	Scenariusz I: Letni upał (Klasyczna klimatyzacja)	7
7.2	Scenariusz II: Ekologiczne chłodzenie (Free Cooling)	7
7.3	Scenariusz III: Blokada deszczowa	7
8	Podsumowanie i wnioski końcowe	8

1 Wstęp i cel projektu

Głównym założeniem projektu było zaprojektowanie oraz zaimplementowanie w języku Python inteligentnego sterownika HVAC opartego na logice rozmytej. Opracowany system rozwiązuje problem wielowymiarowego sterowania mikroklimatem, analizując jednocześnie temperaturę, wilgotność oraz stężenie dwutlenku węgla, wykazując przy tym wyższą elastyczność niż klasyczne regulatory typu on/off.

Zadaniem algorytmu jest utrzymanie rygorystycznego komfortu użytkowników przy jednoczesnej optymalizacji zużycia energii elektrycznej. Sterownik zarządza pracą dwóch urządzeń wykonawczych - klimatyzatora i cyrkulatora, bazując na autorskich regułach rozmytych. Należą do nich m.in. tryb energooszczędnego chłodzenia „Free Cooling” oraz blokada zapobiegająca nawiewowi przy wysokiej wilgotności zewnętrznej.

Silnik wnioskujący został połączony z dedykowanym interfejsem, umożliwiając użytkownikowi podgląd aktywacji reguł i wizualizację zbiorów rozmytych w czasie rzeczywistym.

2 Podstawy teoretyczne logiki rozmytej

2.1 Pojęcie zbioru rozmytego i funkcji przynależności

Klasyczna logika Boole’a opiera się na wartościach dyskretnych $\{0, 1\}$ (fałsz lub prawda). W układach sterowania wymusza to stosowanie ostrych progów zadziałania, co często prowadzi do niestabilności pracy urządzeń (np. ciągle włączanie i wyłączanie sprężarki wokół granicy 25°C).

Logika rozmyta rozwiązuje ten problem, wprowadzając ciągłą funkcję przynależności $\mu(x)$, która przyjmuje wartości z przedziału $[0, 1]$. Zbiór rozmyty A w przestrzeni X definiuje się matematycznie jako zbiór par:

$$A = \{(x, \mu_A(x)) \mid x \in X\}$$

Dzięki temu system płynnie modeluje stany przejściowe. Przykładowo, w opracowanym sterowniku temperatura 24°C nie jest przypisana do jednej kategorii – może jednocześnie należeć do zbioru „optymalna” ze stopniem przynależności $\mu = 0.3$ oraz do zbioru „gorąca” ze stopniem $\mu = 0.7$. Nakładanie się na siebie sąsiednich zbiorów (funkcji przynależności) gwarantuje stabilne i bezskokowe sterowanie mocą silników.

2.2 Operatory logiczne

W systemach logiki rozmytej przesłanki reguł najczęściej łączy się spójnikami logicznymi ORAZ (&) i LUB (|), za które odpowiadają odpowiednio matematyczne T-normy i S-normy.

T-norma (Koniunkcja / AND)

Służy do łączenia warunków, które muszą zachodzić jednocześnie. W klasycznym algorytmie Mamdaniego realizowana jest przez operację minimum:

$$T(\mu_A, \mu_B) = \min(\mu_A, \mu_B)$$

S-norma (Alternatywa / OR)

Służy do łączenia warunków, z których wystarczy spełnienie chociaż jednego. Odpowiada za nią operacja maksimum:

$$S(\mu_A, \mu_B) = \max(\mu_A, \mu_B)$$

Wyznaczanie siły aktywacji reguły (α)

Wynik powyższych operacji wyznacza stopień aktywacji reguły. Parametr ten decyduje, w jakim stopniu dana reguła wpłynie na wyjście, czyli na jakiej wysokości zostanie „ucięty” trójkąt wyjściowy.

Przykład dla sterownika HVAC:

Żałóżmy, że stopień przynależności do zbioru „gorąco” wynosi 0.75, a do zbioru „wysokie CO_2 ” wynosi 0.40. Dla przykładowej reguły:

*JEŚLI gorąco **ORAZ** wysokie CO_2 , THEN klima max*

siła aktywacji wyniesie $\alpha = \min(0.75, 0.40) = 0.40$. Gdyby użyto spójnika **LUB**, siła wyniosłaby $\max(0.75, 0.40) = 0.75$.

2.3 Model wnioskowania Mamdaniego i proces wyostrzania

Ostatnim etapem procesu wnioskowania w logice rozmytej jest wyznaczenie konkretnej wartości sterującej. W zaprojektowanym sterowniku zaimplementowano klasyczny algorytm Mamdaniego, który dzieli się na trzy kluczowe etapy:

1. Implikacja

Aktywowana reguła modyfikuje przypisany jej zbiór wyjściowy proporcjonalnie do obliczonej siły aktywacji α . W modelu Mamdaniego wykorzystuje się do tego operator minimum. Geometrycznie oznacza to, że trójkąt lub trapez reprezentujący wniosek zostaje odcięty w poziomie na wysokości wartości α , tworząc figurę o płaskim wierzchołku.

2. Agregacja reguł

Ponieważ dla danego stanu czujników może aktywować się kilka reguł jednocześnie, ich obcięte figury wyjściowe muszą zostać połączone. Dokonuje się tego za pomocą operatora MAX (S-normy). Figury są na siebie nakładane, tworząc jedną złożoną, pofalowaną bryłę geometryczną.

3. Defuzyfikacja

Ostatnim krokiem jest zamiana zagregowanej figury na konkretną wartość, czyli np. 75% mocy silnika. W systemie zastosowano powszechnie stosowaną metodę środka ciężkości. Wartość wyjściowa y^* obliczana jest ze wzoru:

$$y^* = \frac{\int y \cdot \mu_{agg}(y) dy}{\int \mu_{agg}(y) dy}$$

Gdzie y to zmienna na osi odciętych (zakres mocy), a $\mu_{agg}(y)$ to funkcja przynależności zagregowanej figury. Punkt rzutu środka ciężkości pola pod wykresem na oś poziomą staje się ostatecznym sygnałem sterującym.

3 Projekt struktury sterownika

Architektura opracowanego sterownika oraz definicje zbiorów rozmytych zostały zaimplementowane w dedykowanym module `logika/zmienne.py`. System opiera się na analizie pięciu sygnałów pochodzących z czujników w celu wysterowania dwóch niezależnych urządzeń wykonawczych.

3.1 Definicja przestrzeni rozważań i zmiennych wejściowych

Dla każdego sygnału wejściowego zdefiniowano zakres dopuszczalnych wartości liczbowych oraz przypisano mu trzy zbiory rozmyte. Zestawienie parametrów wejściowych prezentuje Tabela 1.

Tabela 1: Parametry wejściowe systemu

Zmienna wejściowa	Przestrzeń rozważań	Zbiory rozmyte
Temperatura wewnątrz	$15 \div 35$ °C	zimno, optymalnie, gorąco
Wilgotność wewnątrz	$20 \div 80$ %	sucho, norma, wilgotno
Stężenie dwutlenku węgla (CO_2)	$400 \div 2000$ ppm	niskie, średnie, wysokie
Temperatura zewnętrzna	$-20 \div 40$ °C	zimno, optymalnie, gorąco
Wilgotność zewnętrzna	$20 \div 100$ %	sucho, norma, wilgotno

3.2 Definicja zmiennych wyjściowych

Sygnały wyjściowe reprezentują procentoweysterowanie mocy urządzeń wykonawczych. Zrezygnowano z klasycznego sterowania dwustanowego na rzecz płynnej modulacji mocy, co znacząco podnosi kulturę pracy całego układu. Parametry wyjściowe zestawiono w Tabeli 2.

Tabela 2: Parametry wyjściowe systemu

Zmienna wyjściowa	Zakresysterowania	Zbiory rozmyte (Termy)
Cyrkulator (wentylacja / nawiew)	0 ÷ 100 %	stop, średnio, max
Klimatyzator (chłodzenie / osuszanie)	0 ÷ 100 %	stop, średnio, max

4 Baza wiedzy i reguły

Głównym elementem każdego systemu opartego na logice rozmytej jest baza wiedzy, zdefiniowana jako zbiór reguł w postaci implikacji „JEŚLI przesłanka, TO wniosek”. W module `logika/reguly.py` zaimplementowany został zestaw reguł, który rozwiązuje wielowymiarowe konflikty środowiskowe takie jak na przykład jednoczesny wzrost stężenia CO_2 wewnątrz i wystąpienie opadów deszczu na zewnątrz.

Aby dostosować dynamikę układu do rytmu dobowego domowników i ograniczyć hałas w nocy, architekturę wnioskowania podzielono na dwa niezależne profile: dzienny oraz nocny.

Tabela 3: Porównanie reguł sterowania dla profilu dziennego i nocnego

Nr	Warunek	Dzień	Noc
R1	Temp. wew.: gorąco Temp. zew.: zimno Wilg. wew.: sucho/norma Wilg. zew.: sucho/norma	Klima: stop Cyrkulator: max	Klima: stop Cyrkulator: średnio
R2	Temp. wew.: gorąco Temp. zew.: zimno Wilg. wew.: sucho/norma Wilg. zew.: wilgotno	Klima: stop Cyrkulator: średnio	Klima: stop Cyrkulator: stop
R3	Temp. wew.: gorąco Temp. zew.: optymalnie/gorąco lub Wilg. wew.: wilgotno	Klima: max Cyrkulator: max	Klima: średnio Cyrkulator: średnio
R4	Temp. wew.: optymalnie/zimno Wilg. wew.: wilgotno	Klima: średnio Cyrkulator: max	Klima: stop Cyrkulator: średnio
R5	Temp. wew.: optymalnie/zimno CO_2 : średnie/wysokie Temp. zew.: zimno Wilg. wew.: sucho/norma	Klima: stop Cyrkulator: średnio	Klima: stop Cyrkulator: średnio
R6	Temp. wew.: optymalnie/zimno CO_2 : średnie/wysokie Temp. zew.: optymalnie/gorąco Wilg. wew.: sucho/norma Wilg. zew.: sucho/norma	Klima: stop Cyrkulator: max	Klima: stop Cyrkulator: średnio
R7	Temp. wew.: optymalnie/zimno CO_2 : średnie/wysokie Temp. zew.: optymalnie/gorąco Wilg. wew.: sucho/norma Wilg. zew.: wilgotno	Klima: stop Cyrkulator: średnio	Klima: stop Cyrkulator: stop
R8	Temp. wew.: optymalnie/zimno CO_2 : niskie Wilg. wew.: sucho/norma	Klima: stop Cyrkulator: stop	Klima: stop Cyrkulator: stop

4.1 Strategia sterowania w profilu dziennym

Tryb dzienny został zaprojektowany pod kątem maksymalizacji komfortu oraz minimalizacji czasu reakcji układu. W tym wariancie urządzenia wykonawcze pracują w pełnym zakresie. W sytuacji wykrycia skrajnych warunków, takich jak wysoka temperatura lub wysokie CO_2 , silnik rozmyty zezwala na pełną aktywację urządzeń, traktując priorytetowo szybkie przywrócenie parametrów optymalnych.

Idealnym przykładem takiej strategii jest reguła aktywująca klasyczne chłodzenie podczas letniego upału. Dzięki bibliotece `scikit-fuzzy`, przesłanki łączone są w sposób natywny za pomocą operatorów bitowych języka Python – `&` (odpowiadającego T-normie) oraz `|` (odpowiadającego S-normie):

```
ctrl.Rule(temperatura['goraco'] &
          ((temp_zewnetrzna['optymalnie'] | temp_zewnetrzna['goraco']) |
           wilgotnosc['wilgotno']),
          [klima['max'], cyrkulator['max']])
```

4.2 Strategia sterowania w profilu nocnym

Głównym problemem klasycznych układów HVAC pracujących w nocy jest uciążliwy hałas generowany przez klimatyzatory i wentylatory pracujące na wysokich obrotach. W opracowanym sterowniku profil nocny (`reguly_noc`) rozwiązuje ten problem poprzez systemowe nałożenie tzw. kagańca mocy.

Baza wiedzy dla nocy stanowi lustrzane odbicie logiki dziennej, jednak wszystkie wnioski wymuszające agresywną pracę urządzeń zostały celowo zredukowane ze zbioru `max` do poziomu `srednio` (około 50%). Ponadto, jeśli w nocy temperatura utrzymuje się na optymalnym poziomie, a rośnie jedynie wilgotność, system całkowicie blokuje uruchomienie głośnego klimatyzatora na rzecz powolnej wymiany powietrza za pomocą samego cyrkulatora. Takie podejście gwarantuje utrzymanie świeżego mikroklimatu bez ryzyka wybudzenia użytkowników.

Aby zobrazować ten mechanizm, poniżej zestawiono kod odpowiedzialny za osuszanie powietrza. Widać na nim bezpośrednią różnicę w zdefiniowanych wnioskach pomiędzy dniem a nocą, przy zachowaniu identycznych warunków wejściowych:

```
# Tryb dzienny - aktywne osuszanie
ctrl.Rule(
    (temperatura['optymalnie'] | temperatura['zimno']) &
    wilgotnosc['wilgotno'],
    [klima['srednio'], cyrkulator['max']]
)

# Tryb nocny - ciche osuszanie
ctrl.Rule(
    (temperatura['optymalnie'] | temperatura['zimno']) &
    wilgotnosc['wilgotno'],
    [klima['stop'], cyrkulator['srednio']]
)
```

4.3 Algorytmy specjalne: Blokada deszczowa

Klasyczna wentylacja podczas opadów deszczu skutkuje zaciąganiem wilgotnego powietrza z zewnątrz. Aby tego uniknąć, wdrożono algorytm tzw. blokady deszczowej. System przed uruchomieniem wietrzenia każdorazowo analizuje wilgotność zewnętrzną. Jeśli wykryje ulewę, drastycznie ogranicza obroty cyrkulatora, powoli redukując zaduch bez ryzyka zawilgocenia wnętrza.

W kodzie zrealizowano to poprzez rozgałęzienie reguły:

```
# 1. Wentylacja w suchy dzień (pełna moc wymiany)
ctrl.Rule(...) & (co2['wysokie'] | co2['srednie']) &
              wilgotnosc_zewnetrzna['norma'],
              [klima['stop'], cyrkulator['max']])

# 2. Blokada deszczowa (redukcja nawiewu z zewnątrz)
ctrl.Rule(...) & (co2['wysokie'] | co2['srednie']) &
              wilgotnosc_zewnetrzna['wilgotno'],
              [klima['stop'], cyrkulator['srednio']])
```

5 Architektura oprogramowania i implementacja

5.1 Struktura projektu

W celu zapewnienia czytelności oraz łatwości w utrzymaniu kodu, projekt zrealizowano w oparciu o architekturę warstwową. Zamiast jednego, bardzo długiego pliku, kod podzielono na logiczne pakiety, zgodnie z zasadą separacji obaw.

Drzewo katalogów projektu prezentuje się następująco:

```
projekt_hvac/
|-- main.py                # Punkt wejściowy (uruchomienie programu)
|-- interfejs/            # Warstwa prezentacji użytkownika (GUI/HMI)
|   |-- __init__.py
|   '-- okno_glowne.py    # Klasa aplikacji, suwaki wejściowe i wykresy
'-- logika/               # Warstwa matematyczna i decyzyjna
    |-- __init__.py      # Definicja pakietu (wzorzec Fasada)
    |-- zmienne.py       # Definicje przestrzeni rozważań i funkcji
    |-- reguly.py        # Baza wiedzy (tryb dzienny i nocny)
    '-- symulacja.py     # Algorytm wnioskowania i funkcja obliczeniowa
```

5.2 Plik inicjalizacyjny jako wzorzec Fasady

Kluczowym elementem zaimplementowanej architektury jest wykorzystanie specyfiki plików `__init__.py` w języku Python. W folderze `logika` plik ten posłużył do wdrożenia wzorca projektowego Fasada.

Zadaniem Fasady jest ukrycie przed programistą interfejsu skomplikowanej struktury plików silnika rozmytego. Plik `__init__.py` na zapleczu importuje funkcję obliczeniową oraz zbiory z poszczególnych plików, wystawiając je na zewnątrz jako jeden wspólny, łatwy w użyciu pakiet.

Dzięki temu w pliku interfejsu graficznego cała komunikacja z logiką sprowadza się do jednej linijki:

```
from logika import oblicz_moce, temperatura, wilgotnosc, cyrkulator, klima
```

Warstwa wizualna nie musi wiedzieć, gdzie znajdują się reguły, ani w jaki sposób przeliczane są moce. Podaje jedynie stan suwaków do funkcji `oblicz_moce()` i otrzymuje gotowe wyniki, co ułatwia utrzymanie i dalszą rozbudowę projektu.

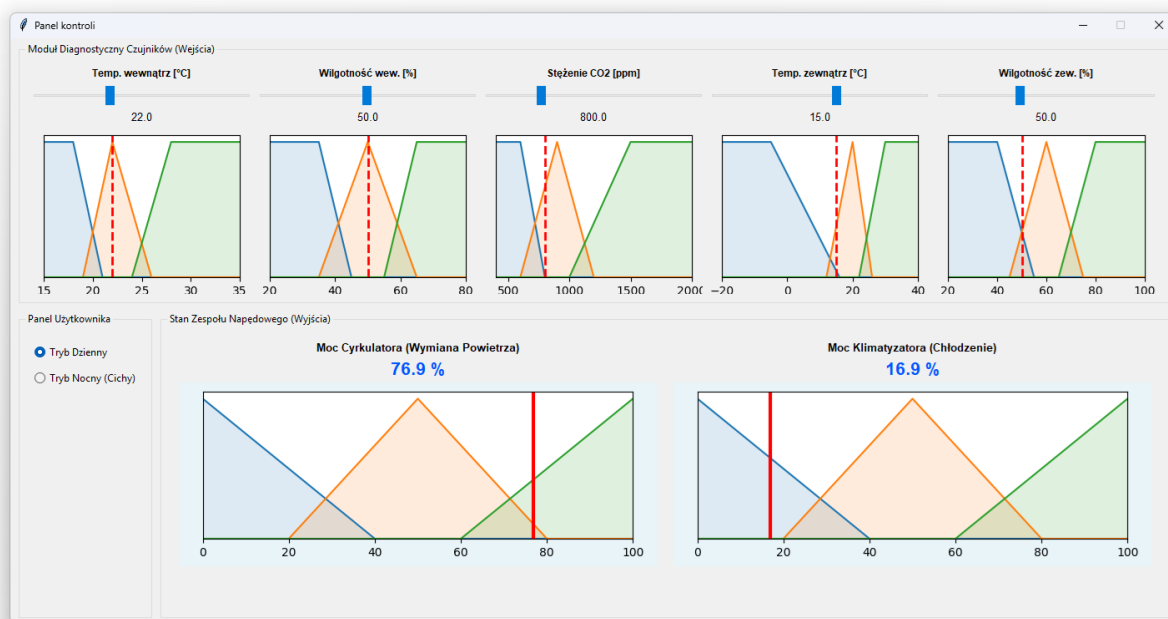
6 Interfejs użytkownika i moduł diagnostyczny

W celu testowania układu stworzono interfejs użytkownika z wykorzystaniem bibliotek `Tkinter` oraz `Matplotlib`. Zbudowana aplikacja pełni funkcję zaawansowanego narzędzia pracującego w czasie rzeczywistym.

Główną zaletą interfejsu jest interaktywna wizualizacja funkcji przynależności. Zmiana parametrów wejściowych za pomocą suwaka odzwierciedlana jest za pomocą przerywanych linii na przypisanych do nich wykresach. Pozwala to użytkownikowi na bieżąco obserwować, w które zbiory rozmyte trafiają aktualne odczyty z symulowanych czujników i jak interpretuje je system.

W dolnej sekcji okna zaimplementowano przełącznik profilu dobowego (Dzień/Noc) oraz wizualizację sygnałów wyjściowych. Wykresy na bieżąco aktualizują ostateczny wynik procesu defuzyfikacji dla cyrkulatora i klimatyzatora, gdzie ostateczną zadaną moc wskazuje gruba, ciągła czerwona linia reprezentująca wyliczony środek ciężkości.

Wygląd opracowanego interfejsu diagnostycznego przedstawiono na Rysunku 1.



Rysunek 1: Interfejs użytkownika aplikacji sterownika.

7 Scenariusze testowe i analiza działania systemu

W celu weryfikacji poprawności zaprojektowanej bazy wiedzy, przeprowadzono symulacje dla skrajnych warunków środowiskowych. Poniżej zaprezentowano trzy kluczowe scenariusze testowe, potwierdzające elastyczność i energooszczędność układu. Ze względu na zastosowanie metody środka ciężkości do defuzyfikacji, zbiór „stop” daje wynik $\approx 13.3\%$, zbiór „średnio” równy 50.0% , a zbiór „max” $\approx 86.6\%$.

7.1 Scenariusz I: Letni upał (Klasyczna klimatyzacja)

Warunki wejściowe: Temp. wew. = 32°C , Temp. zew. = 30°C , Wilg. wew. = 50% , Wilg. zew. = 50% , CO_2 = 800 ppm , Tryb = Dzień.

Wyniki na wyjściu: Klimatyzator $\approx 86.6\%$ (max), Cyrkulator $\approx 86.6\%$ (max).

Analiza: System poprawnie zidentyfikował upał wewnątrz oraz na zewnątrz. Aktywowała się reguła klasycznego chłodzenia,ysterowując oba urządzenia na zbiór „max”, co pozwala na maksymalnie szybkie obniżenie temperatury.

7.2 Scenariusz II: Ekologiczne chłodzenie (Free Cooling)

Warunki wejściowe: Temp. wew. = 32°C , Temp. zew. = 15°C , Wilg. wew. = 50% , Wilg. zew. = 40% , CO_2 = 800 ppm , Tryb = Dzień.

Wyniki na wyjściu: Klimatyzator $\approx 13.3\%$ (stop), Cyrkulator $\approx 86.6\%$ (max).

Analiza: Chociaż w pokoju jest gorąco (32°C), czujnik temperatury zewnętrznej wykrył chłodne powietrze (15°C). Układ poprawnie zablokował sprężarkę klimatyzatora i aktywował wyłącznie cyrkulator na maksymalnych obrotach, wykorzystując chłodne powietrze z zewnątrz do schłodzenia pomieszczenia.

7.3 Scenariusz III: Blokada deszczowa

Warunki wejściowe: Temp. wew. = 22°C , Temp. zew. = 15°C , Wilg. wew. = 50% , CO_2 = 1800 ppm , Wilg. zew. = 95% , Tryb = Dzień.

Wyniki na wyjściu: Klimatyzator $\approx 13.3\%$ (stop), Cyrkulator = 50.0% (średnio).

Analiza: Z powodu krytycznego poziomu CO_2 (1800 ppm), system próbuje wywietrzyć pomieszczenie. Jednak odczyt 95% z czujnika wilgotności zewnętrznej aktywuje algorytm blokady deszczowej. Zamiast otwierać nawiew na 100% , cyrkulator zwalnia do 50% , zapobiegając zaciągnięciu wilgoci i wody do budynku.

8 Podsumowanie i wnioski końcowe

Zrealizowany projekt pokazał, że logika rozmyta dobrze sprawdza się w tego typu zadaniach. Opracowany sterownik z powodzeniem analizuje pięciowymiarową przestrzeń stanów i płynnie zarządza mocą dwóch niezależnych urządzeń wykonawczych.

Zbudowanie analogicznego i równie stabilnego systemu decyzyjnego za pomocą klasycznych, kaskadowych instrukcji warunkowych (`if/else`) byłoby zadaniem niezwykle trudnym. Prowadziłoby to do powstania nieczytelnego i skomplikowanego kodu, silnie podatnego na błędy stanów brzegowych i skoki wartości sterujących. Logika rozmyta pozwoliła na rozwiązanie tego problemu w sposób naturalny, umożliwiając intuicyjne zdefiniowanie wiedzy.